SIMATS ENGINEERING

ASSESSMENT TOOL 1

NAME: GAYATHERI S

REG.NO: 192521306

COURSE NAME: Computer Architecture

COURSE CODE: CSA1202

QUESTIONS:5 Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I GAYATHERI S /192521306 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

GAYATHERI S

Signature of the student

ANNEXURE A

1. Smart City Surveillance System

Analysis

A smart city surveillance system continuously receives video data from multiple cameras. The data is transferred from input devices (cameras) to memory and then processed by the CPU. During peak hours, the huge amount of video data increases traffic on the system bus, causing delays in data transfer and processing.

The interaction among CPU, memory, and I/O devices affects system performance in the following ways:

- **CPU:** Processes video frames and performs threat detection algorithms.
- **Memory:** Stores video frames temporarily before processing.
- **I/O Devices:** Cameras continuously send large amounts of video data.

If the CPU waits for data from memory or I/O devices, processing slows down. Similarly, when multiple devices try to access the same communication path, bus contention occurs, resulting in system lag.

Single-Bus vs Multi-Bus Structure

Single-Bus Structure

- Uses one common bus for CPU, memory, and I/O communication.
- Only one transfer can occur at a time.
- High bus traffic creates bottlenecks.
- Suitable for small systems but inefficient for heavy workloads.

Multi-Bus Structure

- Uses separate buses for CPU, memory, and I/O.
- Multiple data transfers occur simultaneously.
- Reduces waiting time and congestion.
- Provides higher throughput and better scalability.

Improving the Instruction Execution Cycle

System performance can be improved by:

- Using **instruction pipelining** to execute multiple instructions simultaneously.
- Increasing **cache memory** to reduce memory access delays.
- Applying **Direct Memory Access (DMA)** so I/O devices transfer data directly to memory without CPU involvement.
- Using **parallel processing** with multicore processors.
- Implementing **interrupt-driven I/O** instead of polling.

2. Real-Time Stock Trading Application

Analysis

A stock trading application executes millions of instructions every second. During high traffic, execution time increases because of a high **Cycles Per Instruction (CPI)** caused by frequent memory accesses and branch instructions.

The CPU follows the **Fetch–Decode–Execute Cycle**:

1. **Fetch**
    - The CPU fetches an instruction from memory.
2. **Decode**
    - The control unit interprets the instruction.
3. **Execute**

- The Arithmetic Logic Unit (ALU) performs the required operation and stores the result.

Frequent memory access increases fetch time, while branch instructions interrupt the normal execution flow, causing pipeline stalls.

Impact on Execution Time

CPU execution time is given by:

CPU Execution Time = Instruction Count × CPI × Clock Cycle Time

Higher CPI results in increased execution time and slower transaction processing.

Methods to Reduce CPI

1. **Increase Cache Memory**: Reduces memory access time.
2. **Branch Prediction**: Predicts branch outcomes to reduce pipeline stalls.
3. **Instruction Pipelining**: Executes multiple instructions simultaneously.
4. **Out-of-Order Execution**: Executes independent instructions while waiting for slower ones.
5. **Superscalar Architecture**: Executes multiple instructions per clock cycle.
6. **Prefetching**: Loads instructions before they are required.

3. 8085 Microprocessor in an Industrial Temperature Control Unit

Analysis

An industrial temperature control unit uses an **8085 microprocessor** to read sensor values and control actuators. Incorrect temperature readings may occur due to improper use of addressing modes.

Addressing Modes in 8085

1. Immediate Addressing

- Data is included directly in the instruction.
- Example:

```
MVI A, 25H
```

2. Register Addressing

- Data is stored in CPU registers.
- Example:

```
MOV A, B
```

3. Direct Addressing

- Memory address is specified directly.
- Example:

```
LDA 2500H
```

4. Register Indirect Addressing

- Register pair contains the memory address.
- Example:

```
MOV A, M
```

5. Implied Addressing

- Operand is implied by the instruction.
- Example:

```
CMA
```

Possible Causes of Errors

- Incorrect memory addresses.
- Wrong register pair selection.
- Improper initialization of registers.
- Sensor values stored at incorrect locations.
- Incorrect use of indirect addressing.

Using the 8085 Instruction Set Effectively

Useful instructions include:

- **LDA** – Load accumulator from memory.
- **STA** – Store accumulator into memory.
- **MOV** – Transfer data between registers.
- **MVI** – Load immediate data.
- **IN** – Read sensor input.
- **OUT** – Send output to actuators.
- **CMP** – Compare temperature values.
- **JZ**, **JNZ**, **JC** – Conditional branching.
- **CALL** and **RET** – Execute subroutines.

4. Memory Segmentation in 8086 Microprocessor

Analysis

The **8086 microprocessor** uses segmented memory to access up to **1 MB** of memory using 16-bit registers.

Memory is divided into four main segments:

1. Code Segment (CS)

- Stores program instructions.
- Uses the **Instruction Pointer (IP)**.
- Responsible for instruction fetching.

2. Data Segment (DS)

- Stores variables and data.
- Accessed using registers like BX, SI, and DI.

3. Stack Segment (SS)

- Stores function calls, return addresses, and temporary data.
- Uses Stack Pointer (SP) and Base Pointer (BP).

4. Extra Segment (ES)

- Used for additional data storage and string operations.

Address Calculation

Physical Address:

Physical Address = Segment × 16 + Offset

Example:

Segment = 2000H

Offset = 3000H

Physical Address = 23000H

Problems Caused by Improper Segment Handling

- Incorrect segment register values access wrong memory locations.

- Stack overflow occurs if excessive data is pushed onto the stack.
- Invalid memory references cause incorrect program execution.
- Corrupted return addresses result in crashes.

Managing Instruction Execution

Proper execution requires:

- Correct initialization of CS, DS, SS, and ES.
- Proper stack management using PUSH and POP.
- Correct addressing modes.
- Balanced CALL and RET instructions.
- Accurate offset calculations.

Addressing Modes in 8086

- Immediate
- Register
- Direct
- Register Indirect
- Based
- Indexed
- Based Indexed
- Relative

5. Number System Conversion in Data Communication Systems

Analysis

Data communication systems commonly represent packet information in **hexadecimal** because it is compact and easy to read. Internally, however, the CPU processes all data in **binary**.

Importance of Hexadecimal to Binary Conversion

- One hexadecimal digit represents **4 binary bits**.
- Simplifies debugging.
- Reduces representation errors.
- Makes packet analysis easier.
- Matches the CPU's binary processing format.

Example:

Hexadecimal:

```
3A
```

Binary:

```
00111010
```

Effects of Incorrect Conversion

Incorrect conversion may result in:

- Wrong instruction execution.
- Invalid packet interpretation.
- Data corruption.
- Communication failure.
- Incorrect memory addresses.
- Software bugs.
- System crashes.

Role of Efficient CPU Design

Modern CPUs reduce such errors through:

- Error detection mechanisms.
- Efficient instruction decoding.
- High-speed ALUs.

- Cache memory.
- Pipeline processing.
- Exception handling.

Benchmarking

Benchmarking measures system performance using metrics such as:

- CPU execution time.
- Instructions Per Second (IPS).
- CPI (Cycles Per Instruction).
- Throughput.
- Memory latency.

Benchmarking helps identify performance bottlenecks and verifies whether data conversions and instruction execution are functioning correctly.